spendwise .

PROJECT DOCUMENTATION

PERSONAL FINANCE, MADE CLEAR

Make every taka count.

SpendWise is an offline-first expense tracker and personal finance workspace for students — built for the web with Next.js and shipped to Android with Capacitor, backed by Supabase.

Next.js 15 · React 19 | Supabase Auth + Postgres | Capacitor 8 · Android | Offline-first · IndexedDB | BDT (৳)

DOCUMENT	VERSION	PLATFORMS	FEATURES COVERED
Project Summary & Feature Guide	1.0 November 5, 2026	Web + Android Responsive UI	35 documented 7 modules

SPENDWISE · SOURCE-BASED PROJECT REPORT

OVERVIEW

Project at a glance

A student-focused money workspace that records spending, plans budgets, explains habits, and keeps working when the internet does not.

01	Project at a glance	Context	02	Technology & architecture	How it is built
03	Feature inventory	35 features	04	Feature details	What each one does
05	Data model & security	Storage & RLS	06	Offline-first & sync	Local-first behavior
07	Android distribution	Capacitor build	08	Quality & testing	Automated checks
09	Limitations & next steps	Honest scope	10	Quick start	Run it yourself

What SpendWise is

SpendWise is a personal finance and expense-tracking application designed around everyday student spending in Bangladesh. It records income and expenses in Bangladeshi taka (BDT), tracks daily and monthly budgets, turns transactions into charts and reports, and bundles practical tools: loans to repay or money lent, recurring bills, shared expense splits, monthly summaries, and full-account backup and restore.

It runs as a **responsive web app** built with Next.js, React, and TypeScript, and as an **Android app** packaged with Capacitor. **Supabase** supplies authentication and cloud PostgreSQL storage. The app is deliberately **offline-first**: each signed-in account has a local snapshot in IndexedDB, edits are shown immediately, queued, and synchronized when a connection returns.

At a glance

PRODUCT **SpendWise** Student finance tracker	PLATFORMS **Web + Android** One shared codebase	CURRENCY **BDT ৳** Two-decimal precision	FEATURES **35** Across 7 modules
CLOUD **Supabase** Auth + Postgres + RLS	LOCAL DATA **IndexedDB** Per-account cache + queue	ANDROID **SDK 24–36** Java 21 build	TESTS **8 suites** Vitest + static RLS

Who it is for

SpendWise is aimed at students and young adults who want a lightweight, friendly way to understand daily cash flow. The interface is written for that audience — “Student account”, “Pocket money”, campus-oriented categories, and a calm, low-pressure visual tone. The default time zone is **Asia/Dhaka**, and the only supported currency is BDT.

Status: the application code, database migrations, and Android wrapper are present. To use it with real data, you must create your own Supabase project, apply the three migrations, set the two public environment variables, and verify Row Level Security with two real accounts. The repository alone does not connect to a Supabase project.

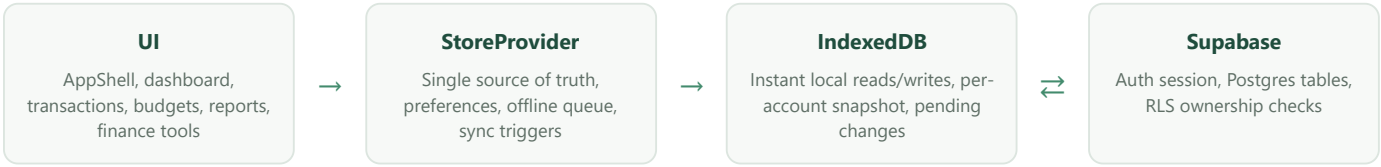
02 · HOW IT IS BUILT

Technology & architecture

A single TypeScript codebase serves two targets: a standard Next.js server build for the website, and a static export embedded inside the Android APK. npm run build produces the server build; npm run build:android sets **CAPACITOR_BUILD=1** and writes the export to **out/**, which is synced into the Android project and should not be deployed as the website.

LAYER	TECHNOLOGY	ROLE IN SPENDWISE
Presentation	Next.js 15 App Router, React 19, TypeScript 5.7	Routing, protected workspace, forms, feature pages, type safety.
Styling & charts	Tailwind CSS 3.4 + custom CSS, lucide-react, Recharts	Responsive app shell, icons, area chart, donut breakdown, progress bars.
Auth & cloud data	Supabase Auth, PostgreSQL, Row Level Security; @supabase/supabase-js 2.89, @supabase/ssr 0.8	Email/password accounts, profile, six owner-scoped tables, cloud sync.
Offline layer	IndexedDB database spendwise-offline, localStorage account pointer	Per-account snapshot, pending change queue, offline session restore.
Android	Capacitor 8: app, browser, filesystem, keyboard, local-notifications, network, share, splash-screen, status-bar	Native shell, deep links, reminders, share sheet, back handling, status bar.
Quality	Vitest 4, ESLint 9, tsc --noEmit, Next build	Unit tests, static policy checks, type checking, linting, production build.

Runtime architecture



The Capacitor plugin layer sits beside the store: network status drives sync, local notifications deliver reminders, and filesystem + share handle CSV/JSON export on Android.

Repository map

PATH	WHAT LIVES THERE
app/	Routes for dashboard, transactions, budgets, reports, loans, shared expenses, monthly summary, backup, settings, profile, notifications, login, signup, auth confirmation, about, contact.
components/	app-shell.tsx (workspace UI), store.tsx (state + sync), new-features.tsx (loans, bills, splits, summary, backup), auth-form.tsx, native-runtime.tsx.
lib/	Calculations, date/time-zone helpers, data access, offline storage, backup, CSV export, reminders, validation, search, auth routing, contact mailto.
supabase/migrations/	Three SQL migrations: initial schema + RLS, profile names, finance feature tables.
types/	Shared TypeScript models for transactions, budgets, loans, bills, shared expenses, preferences.
tests/	Eight Vitest suites covering calculations, dates, backup, offline replay, finance helpers, auth routing, contact, budget selection, static RLS checks.
android/	Generated Capacitor Android project, Gradle build, app id com.yourdomain.spendwise.

03 · FEATURE INVENTORY

Everything the app can do

Thirty-five features across seven modules. “Implemented” means the path exists in the current source; “Partial” means the feature works but has a documented boundary.

#	FEATURE	MODULE	STATUS
ACCOUNTS & ACCESS			
01	**Email/password registration** with first and last name	Accounts	IMPLEMENTED
02	**Sign-in with show/hide password**	Accounts	IMPLEMENTED
03	**Email confirmation** for web and Android deep links	Accounts	IMPLEMENTED
04	**Protected workspace** and session handling	Accounts	IMPLEMENTED
05	**Offline session restore** after a previous sign-in	Accounts	IMPLEMENTED
DASHBOARD & INSIGHTS			
06	**Overview dashboard** with selectable date	Dashboard	IMPLEMENTED
07	**Seven-day spending activity** area chart	Dashboard	IMPLEMENTED
08	**Spending breakdown** donut by category	Dashboard	IMPLEMENTED
09	**Recent transactions** and budget check-in card	Dashboard	IMPLEMENTED
10	**Spending insights** for daily, weekly, monthly periods	Reports	IMPLEMENTED
11	**Monthly summary** with income, expenses, net, categories	Reports	IMPLEMENTED
TRANSACTIONS			
12	**Add income or expense** with categories and payment methods	Transactions	IMPLEMENTED
13	**Edit and delete** transactions with confirmation	Transactions	IMPLEMENTED
14	**Search** across five fields	Transactions	IMPLEMENTED
15	**Filters and sorting** with day grouping	Transactions	IMPLEMENTED
16	**CSV export** of the current filtered view	Transactions	IMPLEMENTED
BUDGETS			
17	**Create, edit, delete budgets** — daily or monthly, optional category	Budgets	IMPLEMENTED
18	**Budget progress** with on-track, warning, and exceeded states	Budgets	IMPLEMENTED
19	**Default daily budget** fallback and latest-budget selection	Budgets	IMPLEMENTED
FINANCE TOOLS			
20	**Loans to repay** and **money lent**	Finance tools	IMPLEMENTED
21	**Partial loan payments** and balance settlement	Finance tools	IMPLEMENTED
22	**Recurring bills** — weekly, monthly, yearly	Finance tools	IMPLEMENTED

#	FEATURE	MODULE	STATUS
23	**Mark bill paid** — creates an expense and advances the date	Finance tools	IMPLEMENTED
24	**Shared expense splits** with settlement tracking	Finance tools	IMPLEMENTED
25	**Android local reminders** one day before a due date	Finance tools	IMPLEMENTED
DATA & PERSONALIZATION			
26	**Full JSON backup export** of records and preferences	Data	IMPLEMENTED
27	**Backup restore** with validation and merge-by-ID	Data	IMPLEMENTED
28	**Settings** — theme, week start, daily budget, payment method, notifications, time zone	Personalization	IMPLEMENTED
29	**Profile** — display name, email, sign out	Personalization	IMPLEMENTED
PLATFORM & SUPPORT			
30	**Offline-first storage** and queued background sync	Platform	IMPLEMENTED
31	**Manual refresh** from the workspace top bar	Platform	IMPLEMENTED
32	**Android native shell** — status bar, splash, back, keyboard, links, share	Platform	IMPLEMENTED
33	**Contact us** form that opens the user's email app	Support	IMPLEMENTED
34	**About page** with personal story and education	Support	IMPLEMENTED
35	**Notifications page** — in-app notification centre	Support	PLACEHOLDER

Reading the status: the notifications page is intentionally an empty state — in-app notification delivery is not implemented. Android loan and bill reminders are a separate, working feature. See section 09 for the full limitations list.

04 · FEATURE DETAILS

Accounts & access

A password-based account system built on Supabase Auth, with a web confirmation flow and a dedicated Android deep link.

01 Email/password registration with identity

IMPLEMENTED

The sign-up form collects first name, last name, email, and a password of at least six characters. Both names are required before the request is sent.

- **Stored metadata:** Supabase receives first_name, last_name, and a combined full_name.
- **Profile creation:** a database trigger creates the profile row and copies the sign-up name, truncated to 100 characters.
- **Confirmation redirect:** web signs up redirect to /auth/confirm; Android uses spendwise://auth/confirm.
- **No duplicate-name logic:** names are display data, not unique identifiers — the email is the account key.

02 Sign-in with show/hide password

IMPLEMENTED

Returning users sign in with email and password. A password-visibility control switches between masked and plain text and announces its state to assistive technology.

- **Safe redirect:** an optional ?next= target is accepted only when it starts with a single / — protocol-relative and backslash paths are rejected.
- **Error handling:** failures are shown inline; verification and credential problems are not leaked as account-existence details.
- **Session:** Supabase manages the session; the app routes to the dashboard on success.

03 Email confirmation for web and Android

IMPLEMENTED

The /auth/confirm page accepts every confirmation shape Supabase can return and exchanges it for a session.

- **PKCE code:** ?code= is exchanged with exchangeCodeForSession.
- **Token hash:** ?token_hash=&type= is verified with verifyOtp for signup, invite, magic link, recovery, email change, and email.
- **Hash tokens:** access and refresh tokens in the URL fragment are installed with setSession.
- **URL hygiene:** after success the query/hash is removed with history.replaceState.
- **Android:** the native runtime catches spendwise://auth/confirm links and forwards them to the confirmation page.

04 Protected workspace and session handling

IMPLEMENTED

All personal pages are guarded in the client store. If no valid session exists, the user is redirected to sign-in with a reason code and personal data is cleared from memory.

- **Auth listener:** onAuthStateChange reacts to sign-in and sign-out, clearing state and protecting against account switching.
- **Defense in depth:** UI guards are convenience only — **Row Level Security is the real boundary** for every read and write.
- **Session timeout:** session checks are wrapped with a timeout so a stalled network call does not hang the workspace.

05 Offline session restore

IMPLEMENTED

After a successful sign-in, the account ID is remembered locally. If the app is opened without a connection, the cached account is restored instead of forcing a login that cannot complete.

- **Conditions:** a remembered account, a readable local snapshot, and a network/session failure. First sign-in still requires the internet.
- **Sign-out:** clears the remembered account pointer, so a shared device does not reopen the previous workspace.
- **Expired sessions:** canRestoreOfflineSession allows offline access while the device is disconnected, while online failures still require a valid session.

04 · FEATURE DETAILS

Dashboard & insights

The first screen answers four questions: what do I have, what did I spend today, how is my budget doing, and where is the money going?

06 Overview dashboard with selectable date

IMPLEMENTED

A time-aware greeting, a date picker, and four headline statistics make up the top of the workspace.

- **Current balance:** all income minus all expenses, considering only transactions at or before “now”.
- **Today’s expenses:** spending on the selected date, not merely the calendar date.
- **Budget remaining:** the most recently created budget (daily or monthly, with its category filter) or the profile default.
- **Monthly expenses:** the selected month’s total, future transactions excluded.
- **Quick actions:** “Add Expense” and “Add Income” open the same form in the correct mode.

07 Seven-day spending activity chart

IMPLEMENTED

A Recharts area chart shows the last seven days ending on the selected date, with a total for that window and a smooth green gradient.

The chart is generated from local transaction data, so it still renders offline; values are recalculated when the date changes.

08 Spending breakdown donut

IMPLEMENTED

A donut chart breaks the selected month’s expenses down by category, with the total spent in the centre and a four-item legend showing each category’s percentage.

Categories use consistent colours and icons — Transport, Food & Drinks, Education, Mobile & Internet, Shopping, Personal Care, Entertainment, Cigarettes, Other.

09 Recent transactions and budget check-in

IMPLEMENTED

The bottom row combines a five-item “Recent transactions” list with a larger budget card.

- **Recent list:** latest five records, with edit and delete actions inline and a link to the full transaction page.
- **Budget card:** spent vs. limit, progress bar, percentage used, money left or over, and a contextual message.
- **Tone:** encouraging when on track, cautionary near the limit, and direct when over budget.

10 Spending insights for three periods

IMPLEMENTED

The Reports page compares the current daily, weekly, or monthly period with the immediately preceding one.

- **Four statistics:** total expenses, total income, average daily spending, and the previous period's expenses.
- **Change indicator:** the percentage more or less than the previous period, or "No previous spending".
- **Category ranking:** each category with an amount, a share-of-spending bar, and a percentage.
- **Insight card:** names the biggest category and its share, then shows income, expenses, and net for the period.

11 Monthly summary

IMPLEMENTED

A focused calendar-month report with a month picker, independent of the dashboard date.

- **Headline figures:** income, expenses, net change, and transaction count.
- **Category totals:** sorted descending with proportional bars relative to total expenses.
- **Accurate boundaries:** uses the first and last day of the selected calendar month, including leap years.

04 · FEATURE DETAILS

Transactions

The core ledger: enter money in and out quickly, find any record, and take the filtered view with you as a spreadsheet.

12 Add income or expense

IMPLEMENTED

One form handles both directions, with validation before anything is saved.

- **Amount:** required, greater than zero, at most two decimals; quick buttons for ৳10, ৳20, ৳50, and ৳100.
- **Type:** Expense or Income. Switching to income suggests "Pocket money"; switching back restores "Food & Drinks".
- **Category:** Transport, Food & Drinks, Education, Mobile & Internet, Shopping, Personal Care, Entertainment, Cigarettes, Other; income adds Pocket money.
- **Subcategory:** optional — Bus, Rickshaw, CNG, Tea, Snacks, Lunch, Dinner, Photocopy, Mobile data, Cigarettes.
- **Payment method:** cash, bKash, Nagad, bank, or other. The preferred method is preselected from settings.
- **Description, date/time, and notes:** optional description and notes; the date and time are converted from the chosen time zone to an absolute timestamp.
- **Errors:** invalid amounts, missing dates, and nonexistent daylight-saving wall times are rejected with a readable message.

13 Edit and delete with confirmation

IMPLEMENTED

Every row exposes edit and delete controls. Editing opens the same form prefilled; deleting opens a modal that names the transaction and warns that removal is permanent. Both actions update the local snapshot first and queue a sync.

14 Search across five fields

IMPLEMENTED

The search box performs a case-insensitive substring match across **category, subcategory, description, notes, and payment method**. This means a search for "class" can find a note, and "bkash" finds the payment method, without a separate mode.

15 Filters, sorting, and day grouping

IMPLEMENTED

- **Filters:** category, payment method, type (expenses/income), and a from/to date range.
- **Sorting:** newest first, oldest first, or highest amount.
- **Grouping:** results are grouped by day with a heading, "Today" where relevant, and a daily expense/income subtotal.
- **Result count:** the header reports how many transactions match, with a helpful empty state when nothing does.

16 CSV export of the filtered view

IMPLEMENTED

The export button writes the **current filtered and sorted list**, not all records. Columns: Date, Type, Category, Subcategory, Description, Payment method, Amount (BDT), Notes.

- **Safe output:** values are quoted and escaped; leading formula characters (=, +, @, -) are neutralised to reduce spreadsheet injection.
- **Web:** downloads spendwise-transactions.csv.
- **Android:** writes to the app cache and opens the native share sheet.

04 · FEATURE DETAILS

Budgets

Simple limits that answer "Can I still spend?" without requiring a full envelope-budgeting system.

17 Daily and monthly budgets with optional category

IMPLEMENTED

Each budget has a name, an amount, a period (daily or monthly), and an optional category filter. A budget named "Food & Drinks" with that category only counts food spending; leaving the category empty counts everything.

- **Create:** name and positive amount are required; period defaults to monthly.
- **Edit:** all fields can change; the original record is updated by ID.
- **Delete:** removes the budget without touching transactions; the default daily budget remains in settings.
- **Database support:** a month-specific budget column exists and is preserved on edit, but budgets created in the current UI are recurring.

18 Progress states and remaining amount

IMPLEMENTED

Each card shows spent vs. limit, a progress bar, the percentage used, and the remaining or over-budget amount.

- **On track:** below 80% of the limit.
- **Approaching limit:** 80–99%, with a caution colour.
- **Budget exceeded:** 100% or more, with the over-budget amount.

19 Default daily budget and latest-budget selection

IMPLEMENTED

The overview card chooses the **most recently created budget** and respects its period and category. If no budgets exist, it falls back to the profile's default daily budget (৳300 for a new account).

- **What counts:** only transactions at or before "now" — future-dated records do not inflate spending.
- **Daily budgets:** count the selected day; monthly budgets count the selected month.
- **Category budgets:** count only that category, so a food budget does not absorb transport costs.

Design choice: budgets are intentionally lightweight. There is no rollover, envelope transfer, or per-week budget in the current model; the period choices are daily and monthly.

04 · FEATURE DETAILS

Finance tools

A hub for the money that is not a simple transaction: loans, recurring bills, shared costs, and monthly review.

The **Finance tools** page links four tools: Loans & bills, Shared expenses, Monthly summary, and Backup & export. Each opens a focused page with its own header and navigation back to the workspace.

20 Loans to repay and money lent

IMPLEMENTED

Track two directions: “I need to repay” and “I lent money”. Each record stores the other person, principal, remaining balance, due date, and optional notes.

- **Balance display:** remaining and principal, e.g. “৳700 remaining of ৳1,000 · due 2026-12-01”.
- **Independent ledger:** loan movements do **not** change the transaction history or current balance — by design.
- **Editing:** changing the principal preserves the amount already paid and recalculates the remaining balance.

21 Partial loan payments and settlement

IMPLEMENTED

A compact payment field accepts any valid amount up to the remaining balance. On update, the remaining balance drops, and the list refreshes immediately.

- **Validation:** positive amount, at most two decimals, never more than what is owed.
- **Settlement:** at zero remaining, the reminder is cancelled and the payment field disappears.
- **Reminders:** rescheduled against the due date after a partial payment, if notifications are on.
- **Deletion:** removes the loan and cancels its scheduled reminder after confirmation.

22 Recurring bills

IMPLEMENTED

Repeat a bill weekly, monthly, or yearly with a name, amount, category, next due date, and a “remind me one day before” toggle.

- **Monthly dates:** the 31st clamps correctly to shorter months; month-end bills stay at month-end.
- **Yearly dates:** February 29 becomes February 28 in non-leap years.
- **List view:** amount, frequency, next due date, and reminder state.

23 Mark bill paid

IMPLEMENTED

One action records the expense and moves the bill forward.

- **Expense created:** type expense, the bill’s amount and category, description “Bill payment: <name>”, preferred payment method, current timestamp.
- **Next due date:** calculated strictly after today, advancing repeatedly if the bill is overdue.
- **Atomic local write:** the bill update and the transaction are queued together before syncing.
- **Reminder:** rescheduled for the new due date.

24 Shared expense splits

IMPLEMENTED

Split a cost with people identified only by name — they do not need SpendWise accounts and are not notified.

- **Input:** title, total, and comma-separated participants (up to 100 other people).
- **Equal split:** the total is divided in integer paisa; any remainder is distributed one paisa at a time, so the shares always sum to the total.
- **Your share:** excluded from what others owe you; duplicates and your own display name are rejected.
- **Settlement:** each participant can be toggled "Settled" or "Mark settled"; outstanding is owed minus settled.
- **Privacy:** the split is a private record in your account, not a shared document.

25 Android local reminders

IMPLEMENTED

On Android, loans and recurring bills can schedule a device notification for about **9:00 AM the day before** the due date.

- **Permission:** the app checks and requests notification permission; reminders cannot be scheduled without it.
- **Preference:** the global notification switch enables or disables all loan and bill reminders; each bill also has its own toggle.
- **Rescheduling:** saving, paying, editing, or deleting a record cancels and re-creates the relevant notification.
- **Boundary:** reminders only work in the native Android app; the web build does not deliver notifications, and Android may delay delivery.

04 · FEATURE DETAILS

Data & personalization

Your records should be portable and the workspace should feel familiar. These features cover backup, restore, preferences, and identity.

26 Full JSON backup export

IMPLEMENTED

"Export full backup" writes a structured JSON file containing the account ID, all transactions, budgets, loans, recurring bills, shared expense splits, and preferences.

- **Format:** { format: "spendwise-backup", version: 1, data: { … } }, pretty-printed.
- **Delivery:** browser download on the web; cache file plus native share sheet on Android.
- **Clarity:** the page states that the file is not encrypted and should be stored privately.

27 Backup restore with validation and merge

IMPLEMENTED

Restore is deliberately strict and non-destructive.

- **Schema validation:** checks the format marker, version, record shapes, UUIDs, dates, money limits, and allowed enum values before importing.
- **Account match:** the backup's user ID must equal the signed-in account ID.
- **Merge by ID:** records whose IDs already exist are kept as-is; only new IDs are added. Nothing is overwritten or deleted.
- **Sync:** added records enter the pending queue and reach Supabase when online.
- **Reminders:** upcoming loan and bill reminders are re-scheduled, with a warning if permission or a future time is missing.

28 Settings and preferences

IMPLEMENTED

Preferences are saved to the profile, applied immediately, and synced like any other record.

- **Currency:** BDT is shown and fixed; the app is single-currency by design.
- **Theme:** Light, Dark, or System. System follows the device's colour-scheme preference live.
- **Week starts on:** Sunday or Monday.
- **Default daily budget:** the fallback used when no budgets exist.
- **Preferred payment method:** preselected in the transaction form.
- **Notification preference:** enables or disables loan and bill reminders.
- **Time zone:** Asia/Dhaka, UTC, Asia/Kolkata, Europe/London, or America/New_York. It controls daily/monthly boundaries and report dates.

29 Profile and sign-out

IMPLEMENTED

The profile page shows initials, the display name, a "Student account" label, and a small account form.

- **Editable:** display name — saved to preferences and synced.
- **Read-only:** email address and preferred currency.
- **Navigation:** shortcut to Settings.
- **Sign out:** clears the local remembered account and workspace state, then returns to sign-in.

04 · FEATURE DETAILS

Platform & support

The parts that make SpendWise feel like an app rather than a form: offline behavior, native integration, and human support pages.

30 Offline-first storage and queued sync

IMPLEMENTED

Every account gets its own record in the spendwise-offline IndexedDB database. Reads come from this snapshot, and writes are applied locally before any network call.

- **Optimistic UI:** creating, editing, or deleting a record updates the screen immediately.
- **Pending queue:** changes are stored with their account ID and replayed in order.
- **Sync triggers:** reconnecting, returning to the Android app, a 30-second retry while changes are pending, and the manual refresh button.
- **Remote refresh:** after flushing pending changes, the app pulls transactions, budgets, loans, bills, shared expenses, and the profile.
- **Account safety:** queued changes are scoped to one account; a different signed-in user cannot replay them.

31 Manual refresh

IMPLEMENTED

The refresh icon in the workspace top bar requests the latest cloud data. While offline it returns without changing local data; queued changes still synchronize automatically after reconnecting. A toast confirms success.

32 Android native shell

IMPLEMENTED

The Capacitor wrapper adds the details that a browser tab cannot provide.

- **Status bar and splash:** light status-bar style with the app's cream background; configured splash screen.
- **Back button:** navigates history when possible; otherwise asks "Exit SpendWise?" before exiting.
- **External links:** http/https links outside the app origin open in the in-app browser.
- **Keyboard:** resize behavior configured for forms.
- **Share and files:** CSV and JSON exports write to cache and open the Android share sheet.
- **Deep links:** spendwise://auth/confirm routes to the confirmation page.
- **Network awareness:** native network status, rather than only browser events, drives sync.

33 Contact us

IMPLEMENTED

A support page for bug reports, account help, suggestions, or anything else. The form collects an optional name and email, a topic, and a message of 10–5,000 characters.

Submitting builds a mailto: link and opens the user's email app with a draft. Nothing is sent by SpendWise itself, and the page says so. A direct email link is also shown as a fallback.

34 About page

IMPLEMENTED

A personal page introducing the developer, Tamim Ahmed — a first-year BBA Honours student at Government Debendra College, Manikganj, and an aspiring developer. It includes a short story, education card, photographs, a motto, and a return link to the workspace.

35 Notifications page (placeholder)

PLACEHOLDER

The page exists and is reachable from settings, but it is an intentional empty state: "No notifications yet — notification delivery is not available yet." It links back to notification preferences.

Android loan and bill reminders are delivered locally and are separate from this in-app notification centre.

05 · STORAGE & PROTECTION

Data model & security

Six PostgreSQL tables, each owned by exactly one authenticated user and protected by Row Level Security.

TABLE	WHAT IT STORES	OWNERSHIP
profiles	Display name, currency (BDT), default daily budget, theme, week start, preferred payment method, notification switch, time zone. Primary key equals the auth user ID.	Owner-only; 4 policies
transactions	Type (income/expense), amount numeric(12,2), category, subcategory, description, payment method, timestamp, notes, timestamps. Indexed by owner/date and owner/category/date.	Owner-only; 4 policies
budgets	Name, amount, period (daily/monthly), optional category, optional month (first day only), created/updated timestamps.	Owner-only; 4 policies
loans	Direction (repay/lent), person, principal, remaining (0 ≤ remaining ≤ principal), due date, notes.	Owner-only; 4 policies
recurring_bills	Name, amount, category, frequency (weekly/monthly/yearly), next due date, reminder flag.	Owner-only; 4 policies
shared_expenses	Title, total, paid-by name, shares as a JSONB array of {name, amount, settled}, created/updated timestamps.	Owner-only; 4 policies

Security controls in the code

Row Level Security everywhere

RLS is enabled on all six tables. Select, insert, update, and delete are each governed by auth.uid() = user_id; updates and inserts also use WITH CHECK so a client-supplied owner cannot reassign a record.

Server-assigned ownership

user_id defaults to auth.uid() at the database. Profiles use the auth user ID as their primary key. Ownership never comes from client metadata.

Safe profile trigger

A SECURITY DEFINER trigger with an empty search path creates the profile and copies only the name from sign-up metadata, truncated to 100 characters. Public execute permission is revoked.

Database constraints

Money is numeric(12,2), amounts must be positive, loan remaining cannot exceed principal, enums are checked, and text lengths are bounded. Indexes support owner/date/category queries.

No privileged key in the client

The frontend uses the publishable/anon key only. A service-role or secret key would bypass RLS and is explicitly excluded from the setup instructions.

Validated inputs and files

Transactions and money are validated in the UI and data layer. CSV output guards against formula injection. Backup restore validates structure, IDs, dates, enums, and account ownership before merging.

Money and time correctness

Paisa precision: the client sums integer paisa and rounds only at the boundary. **Time zones:** daily/monthly boundaries, report dates, and transaction dates use the account's chosen IANA zone, not the server or browser zone. **DST:** nonexistent daylight-saving wall times are rejected, while ambiguous fall-back times resolve to one valid offset. **Future records:** dashboard totals exclude transactions dated after "now".

06 · LOCAL-FIRST BEHAVIOR

Offline-first & sync

SpendWise writes locally first, then syncs. The app never blocks an edit on a network request, and it does not show an offline warning.

The write path

1. **Read the local snapshot**
   The account's IndexedDB state is the source of truth for the current session.
2. **Apply the change locally**
   The new record, edit, or deletion is merged into the snapshot and written back to IndexedDB.
3. **Publish to the UI**
   React state updates immediately, so the dashboard, lists, and totals reflect the change.
4. **Queue the change**
   A pending change with the account ID and operation type is appended to the queue.
5. **Sync when possible**
   If online, the queue is replayed in order through Supabase. Each success removes that change from the queue.
6. **Refresh from the cloud**
   After the queue empties, the app pulls the latest records and profile, then reapplies any remaining pending changes.

Behavior by situation

SITUATION	WHAT HAPPENS
Online edit	Shown instantly; queued; synced in the background. A toast may confirm the action.
Offline edit	Shown instantly; queued; no offline warning. Data remains available after closing and reopening the app.
Reconnect / app resumed	The online listener, native network listener, or Android app-state listener triggers sync. A 30-second retry runs while changes are pending.
Manual refresh	Online: flushes pending changes, then pulls cloud data. Offline: leaves local data untouched.
Sign out	Clears the remembered account and in-memory data. The cached snapshot remains on the device unless app data is cleared.
First sign-in	Requires internet. Offline access only works after a previous successful sign-in and cache.

Conflict and data-safety boundaries

- **Pending changes win locally:** remote data is loaded, then pending operations are reapplied so the local view stays consistent until sync finishes.
- **Account isolation:** pending changes carry a user ID and cannot be replayed into a different account.
- **No real-time multi-tab sync:** two open tabs do not update each other automatically. Reload or use refresh to see another session's changes.
- **Not encrypted locally:** IndexedDB data and exported backups are plain. Clearing app data removes cached records, including changes that have not synced.

07 · MOBILE DELIVERY

Android distribution

The Android app is a Capacitor wrapper around a static Next.js export. The website and the APK share the same UI and data layer.

Build pipeline

1. Static export
npm run build:android

CAPACITOR_BUILD=1 → out/

→

2. Sync
npx cap sync android

copies web assets + plugins

→

3. Gradle build
Android Studio or gradlew assembleDebug

→

4. APK
android/app/build/outputs/apk/

Project configuration

Application ID com.yourdomain.spendwise, minimum SDK 24, target SDK 36. Build with Java 21 (Android Studio's bundled JBR is suitable). Change the application ID before publishing under your own domain.

Supabase redirects

Add https://YOUR-DOMAIN/auth/confirm, https://YOUR-DOMAIN/auth/callback, and spendwise://auth/confirm to the redirect allow list. Set the Site URL to the deployed website origin.

Release signing

Build a signed release APK or Android App Bundle with a keystore stored outside the repository. Keep the same signing key for every update and increment versionCode.

Device test checklist

Install and launch, sign up, confirm email, sign in, restart, sign out; test offline edits and reconnect; test reminders, CSV sharing, deep links, back navigation, and keyboard behavior.

What is native, and what is not

CAPABILITY	BEHAVIOR
Local reminders	Native Android local notifications, scheduled about 9:00 AM the day before. Requires permission; OS may delay delivery.
Share sheet	CSV and JSON exports are written to the app cache and shared through Android's share dialog.
Deep-link confirmation	spendwise://auth/confirm is handled by the native runtime and forwarded to the web confirmation page.
Status bar / splash / keyboard	Configured through Capacitor plugins; light status bar style and a cream splash background.
External links	Non-app http/https links open in the in-app browser instead of replacing the app.
Push notifications	Not implemented. Reminders are local-only. There is no server push or in-app notification feed.

Remember: a website deployment does not update an installed APK. After frontend changes, rebuild the static export, run npx cap sync android, and produce a new APK. Test on a physical device before sharing.

08 · VERIFICATION

Quality & testing

Eight Vitest suites protect the calculations, date handling, offline replay, backup format, and policy declarations. Type checking, linting, and a production build complete the checks.

TEST SUITE	WHAT IT VERIFIES
calculations.test.ts	Income/expense separation, paisa precision, current balance, empty totals, future transactions excluded, time-zone date boundaries, DST rejection, amount validation.
finance-utils.test.ts	Equal cent-safe splitting, weekly/monthly/yearly bill advancement across month-end and leap years, 9 AM prior-day reminder time, five-field search, calendar month bounds, outstanding split balance, merge without overwriting IDs.
offline-storage.test.ts	Offline creates/edits/deletes, pending queue replay, account scoping, immediate local reflection, loans/bills/shared expenses in the queue.
backup.test.ts	Backup creation, parsing, account mismatch rejection, merge-by-ID behavior.
budget-overview.test.ts	Latest-budget selection, fallback ordering, daily category spending, monthly spending, future exclusion.
auth-routing.test.ts	Private route detection, login redirect targets, offline session restore conditions, session timeout behavior.
contact.test.ts	Mailto construction and contact address handling.
rls.test.ts	Static check that all tables enable RLS and declare select/insert/update/delete policies; profile trigger name handling; finance table policies.

Commands

Type check
npm run typecheck

Lint
npm run lint

Tests
npm test

Build
npm run build

What automated tests do not cover

- Live Supabase signup, email delivery, session refresh, and database CRUD.
- Real Row Level Security behavior with two accounts — the SQL test reads the migration and checks that policies exist, but does not execute them.
- Native Android behavior: notification delivery, share sheet, deep links, keyboard, and back navigation.
- **Recommendation:** run the two-account RLS exercise and the device checklist in ANDROID_SETUP.md before using real financial data or distributing an APK.

09 · HONEST SCOPE

Limitations & next steps

The app is coherent and usable, but it is not a finished commercial product. These boundaries are deliberate and documented in the repository.

Current limitations

AREA	STATUS	DETAIL
Password reset	NOT IMPLEMENTED	No forgot-password flow. Confirmation parsing recognizes recovery links, but there is no reset screen.
Account deletion	NOT IMPLEMENTED	Users cannot delete their account or erase cloud data from the UI.
Email change	NOT IMPLEMENTED	The profile email is read-only.
OAuth / social sign-in	NOT IMPLEMENTED	Email and password only.
In-app notifications	PLACEHOLDER	The notifications page is an empty state; no feed or delivery pipeline exists.
Push notifications	NOT IMPLEMENTED	Only local Android reminders for loans and bills.
Real-time multi-tab sync	NOT IMPLEMENTED	Two open tabs do not update each other automatically; reload or refresh.
Encrypted local data / backups	NOT IMPLEMENTED	IndexedDB and JSON backups are plain text; the user must protect the device and files.
Multi-currency	NOT IMPLEMENTED	BDT only, by design and database constraint.
Receipt attachments	NOT IMPLEMENTED	No image or file attachments on transactions.
Recurring income	NOT IMPLEMENTED	Recurring bills are expenses; income is entered manually.
Shared expense invitations	PARTIAL BY DESIGN	Participants are names only; they are not invited or notified and need no account.
Loan and transaction link	PARTIAL BY DESIGN	Loan balances do not create or alter transactions and are excluded from current balance.
Live RLS automation	STATIC TEST ONLY	The automated suite checks the SQL text, not a running Supabase project.

Suggested next steps

Account completeness

Add password reset, authenticated email change, and account deletion with a clear data-erasure confirmation. These are the biggest gaps for real users.

Sync maturity

Add Supabase realtime subscriptions or a version field for conflict detection, so multiple devices and tabs converge without a manual refresh.

Privacy hardening

Encrypt local snapshots and backups with a user passphrase, or at least offer an encrypted export format. Add an app lock or biometric gate.

Notification centre

Turn the placeholder into a real feed for budget alerts, bill due dates, and sync errors, with read/unread state.

Reporting depth

Add trend lines across months, income-vs-expense history, budget-vs-actual charts, and a printable PDF report.

Capture speed

Receipt photos, saved transaction templates, and quickly repeated expenses would reduce friction for daily use.

10 · RUN IT YOURSELF

Quick start

From a fresh clone to a running workspace, plus the commands for the Android build.

Web setup

1. **Install dependencies**
   Node.js 22 or newer, then npm ci.

2. **Create a Supabase project**
   Wait for the database to be ready, then apply the three migrations in order: 20261030000000_initial.sql, 20261030000001_profile_names.sql, 20261104000000_finance_features.sql.

3. **Add environment variables**
   Create .env.local with NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY. Use the publishable/anon key — never a service-role key.

4. **Configure auth URLs**
   Set the Site URL and allow /auth/confirm, /auth/callback, and spendwise://auth/confirm as redirects. For cross-device confirmation emails, use a token-hash link to /auth/confirm.

5. **Run and verify**
   npm run dev, open the landing page, create an account, confirm the email if enabled, and sign in. Then run npm run typecheck, npm run lint, npm test, and npm run build.

Android build

```
npm run build:android
npx cap sync android
npx cap open android
# then Build > Build APK(s), or:
Push-Location android; .\gradlew.bat assembleDebug; Pop-Location
```

The debug APK is written to android\app\build\outputs\apk\debug\app-debug.apk. Use a signed release build for distribution; do not distribute a debug APK.

Environment contract

VARIABLE	PURPOSE
NEXT_PUBLIC_SUPABASE_URL	The Supabase project URL. Public by design.
NEXT_PUBLIC_SUPABASE_ANON_KEY	The publishable or legacy anon key. Public by design; RLS enforces access.
CAPACITOR_BUILD	Set to 1 by the Android build script to produce a static export.

Default new-account state: empty transactions and budgets, a ৳300 daily budget, Asia/Dhaka time zone, system theme, cash as the preferred payment method, and notifications enabled. Add an income through the transaction form to give the account a balance.